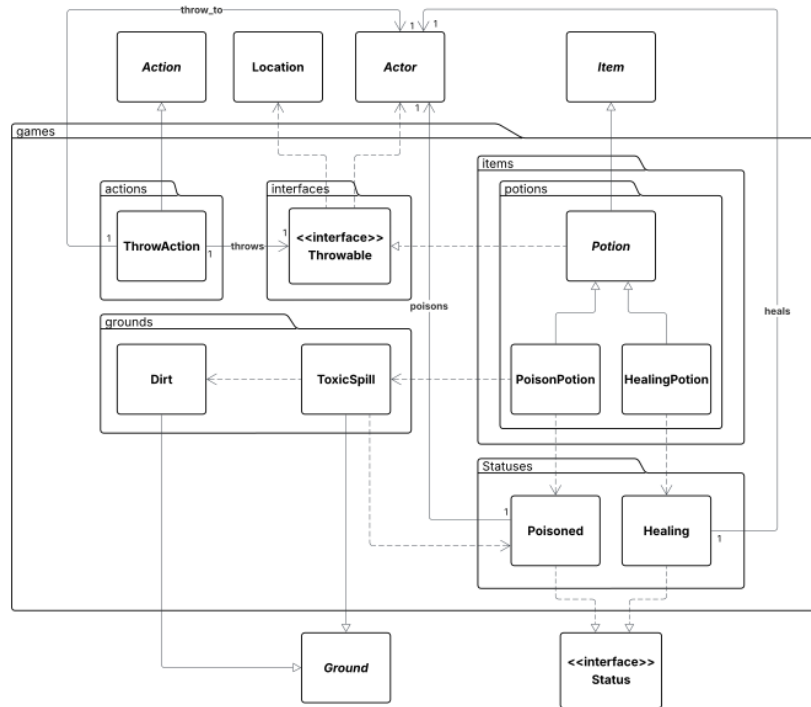
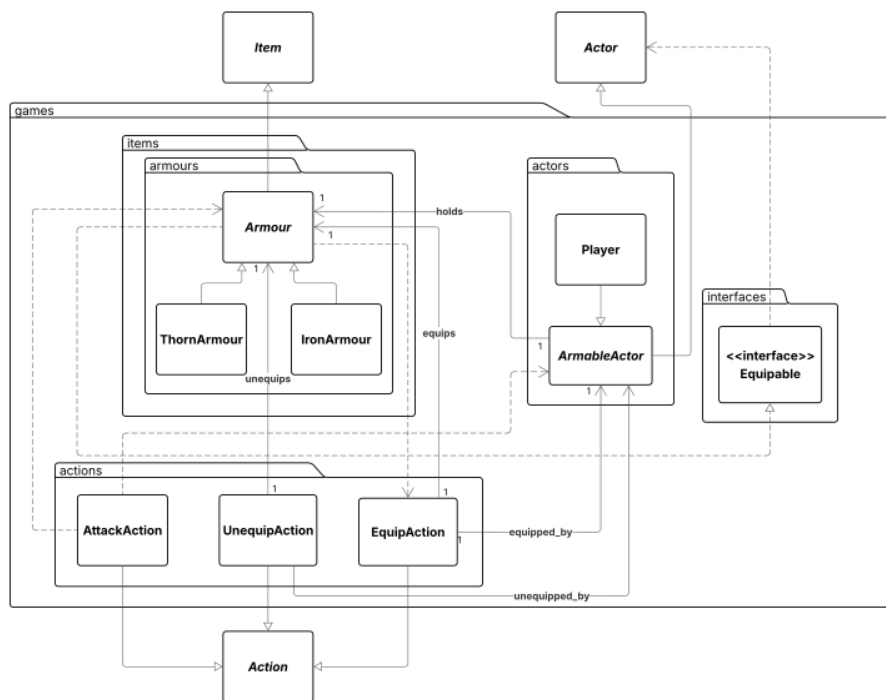


Design Diagrams

REQ 3



REQ 4



Design Rationale

REQ 3

Design

In this requirement, I introduce a potion system where all potions are throwable and interact through a unified Throwable interface. A shared abstract Potion class defines core behaviour such as applying effects. Concrete potion types such as PoisonPotion and HealingPotion extend Potion, each overriding applyEffect() to introduce unique gameplay effects.

To support throwing, the ThrowAction class handles targeting and effect execution. Actor can always throw at himself or other actors that are surrounds him.

For area effects, PoisonPotion additionally generates ToxicSpill ground, which applies ongoing poison effects to any actor stepping on it.

Overall, the design avoids modifying the engine and builds on existing Action and Item extension points.

Advantages of the Design

Single Responsibility Principle (SRP)

Each major component has one clear purpose:

- Potion classes define effect logic only
- ThrowAction handles throw mechanics and targeting
- Status classes (e.g., Poisoned, Healing) manage ongoing effects

This separation improves cohesion and testability.

Dependency Inversion Principle (DIP)

Gameplay behaviour depends on abstractions:

- ThrowAction depends on Throwable instead of specific potions
- Potion effects depend on the Status interface, not concrete actors

This allows future throwable objects (e.g., smoke bombs, freezing flasks) without modifying core logic.

Open-Closed Principle (OCP)

The system is open to extension by adding new potions or status types without changing existing code. To create a new potion, only a subclass of Potion is required.

Likewise, new status effects can be added by implementing the Status interface.

KISS & DRY

Using one universal interaction (throw) avoids introducing multiple action categories such as consume vs throw.

Shared mechanics like effect triggering and targeting are centralised, reducing duplication.

Disadvantages of the Design

Scattered AoE Logic (Potential DRY Issue)

Both PoisonPotion and HealingPotion implement similar throwAt() logic, including iterating surrounding locations and applying effects to nearby actors. If more throwable potions are introduced in the future, this pattern may lead to repeated code across potion subclasses.

Connascence Analysis

- **Connascence of Name** appears through shared method names such as throwAt() and applyEffect(). All potion types follow these naming conventions to guarantee uniform effect triggering and area-of-effect handling. This uniformity enables polymorphic behaviour and ensures each potion integrates cleanly with the throwing system without requiring modifications to ThrowAction.
- **Connascence of Type** is reflected in the shared method signature throwAt(Location, Actor, Actor) and the use of abstract types such as Potion, Throwable, and Actor. By relying on common abstractions rather than concrete potion classes, this approach aligns with the Dependency Inversion Principle, maintaining loose coupling and allowing new potion effects to be introduced without altering existing engine logic.
- **Connascence of Meaning** exists in the ordered parameters (location, target, attacker). The semantic meaning of each argument must be preserved to avoid incorrect effects (e.g., accidentally healing the attacker instead of the target). This dependency is acceptable in this context as the meaning is documented and consistent across all potions.
- **Connascence of Execution Order** is visible inside throwAt(), where removing the potion from inventory must occur before applying the effect, and splash effects must be applied after the primary hit. This explicit sequencing ensures accurate resource consumption and effect propagation, reducing logical errors during gameplay.

Conclusion

All potion types inherit from the Potion abstract class, reducing duplication and improving maintainability (DRY). The use of the Throwable interface ensures that high-level classes depend on abstractions rather than concrete items (DIP), while ThrowAction encapsulates interaction logic in a single place and adheres to the Single Responsibility Principle.

By encapsulating long-term effects within independent Status classes, the design achieves high cohesion and ensures future status types can be introduced without modifying existing functionality (OCP). This throwable-based architecture also offers future extensibility beyond potions, supporting any throwable gameplay object without altering core engine systems.

Overall, this design balances clarity, extensibility, and simplicity, making it well-suited to the engine constraints and scalable for future gameplay expansion.

REQ 4

Design 1

In this design, armour behaviour was embedded directly inside the Player class. The player stored a reference to an equipped armour item and implemented all damage-reduction logic inside Player.hurt(). Armour subclasses (e.g., IronArmour, ThornArmour) exposed a defend() method, and the player would:

1. Check if armour was equipped
2. Call calculateDamage() and defend() on the armour
3. Manually update DEFENSE_LEVEL
4. Traverse the inventory to remove broken armour

This design successfully provided armour support without modifying engine code and allowed special armour effects through polymorphism. It also followed the assignment rule by extending Item and handling defence inside the game layer rather than engine core.

Disadvantages of the Previous Design

Mixed Responsibilities (SRP Violation)

Player.hurt() handled multiple concerns simultaneously:

- HP reduction
- Armour durability management
- Equipment removal
- Inventory manipulation

This merged survival logic, combat logic, and equipment state handling into a single method, reducing cohesion and increasing maintenance cost.

Tight Coupling Between Player and Armour

The player directly managed armour lifecycle, which meant:

- Defence behaviour could not be reused by other actors without copy-pasting logic
- Any new equipment behaviour required modifying the player class
- Testability suffered, since armour logic could not be isolated from Player

Reduced Extensibility

Only Player could wear armour—future NPCs or bosses could not gain armour without duplicating the logic.

This limited scalability once new actor types were introduced.

Inventory Traversal Overhead

Repeated inventory checks to locate equipped armour introduced unnecessary coupling and potential duplication.

As armour effects increased (elemental shields, magical wards, etc.), this pattern risked expanding into more traversal logic.

Design 2

In this requirement, I introduce an armour system that allows certain actors to equip defensive items and reduce incoming damage. To support reusable armour behaviour, an abstract `ArmableActor` class is introduced, which extends `Actor` and contains a reference to the currently equipped `Armour`, along with a defence statistic.

A shared abstract `Armour` class defines core behaviour such as durability tracking and damage mitigation through the `calculateDamage()` method. Concrete armour types such as `IronArmour` and `ThornArmour` extend `Armour`, each overriding or extending behaviour to introduce unique effects (e.g., damage reflection from thorns).

To manage equipping and unequipping, two dedicated actions (`EquipAction` and `UnequipAction`) are implemented. These actions interact only with the high-level abstractions `ArmableActor` and `Armour`, enabling the system to equip armour without depending on specific armour types or concrete player classes.

Overall, this design integrates smoothly with the existing action and actor framework, avoids engine modification, and ensures any armour-wearable actors can gain defence through a shared, extensible mechanism.

Advantages of the Design

Single Responsibility Principle (SRP)

Each major component fulfils one clear responsibility:

- `ArmableActor` manages wearing armour and delegating damage reduction
- `Armour` classes define defence and durability logic
- `EquipAction/UnequipAction` handle interaction and equipment state changes

This separation improves cohesion, keeps armour logic out of unrelated classes, and simplifies maintenance and testing.

Open–Closed Principle (OCP)

The armour system is open for extension without modifying existing code:

- New armour variants can be introduced by subclassing `Armour`
- Future actors that can wear equipment only need to extend `ArmableActor`

- Special effects like thorns or elemental resistance can be added without altering action logic

The system supports scalability and new armour mechanics with minimal change to existing components.

Dependency Inversion Principle (DIP)

Gameplay logic depends on abstractions rather than concrete classes:

- EquipAction and UnequipAction operate on the ArmableActor and Armour abstractions
- Combat logic calls armour.calculateDamage() without knowing the armour type

This reduces coupling and allows mock armour objects to be used in unit testing.

KISS & DRY Principles

- Reusable ArmableActor avoids duplicating armour-wearing rules across multiple characters
- Shared Armour behaviour prevents duplicated durability and defence code
- Simple equip/unequip cycle avoids complex equipment systems not required by the engine

The design is concise, clear, and easy to reason about.

Disadvantages of the Design

Engine-side dependence on runtime casting

Even with ArmableActor abstracting armour functionality, AttackAction must still perform a runtime cast (getActorAs(ArmableActor.class)) to detect whether the target supports armour effects. This introduces a small dependency leak back into combat logic, since the engine action layer needs awareness of a game-specific subclass (ArmableActor) instead of a pure capability interface. As a result, armour effect triggering is not entirely decoupled from attack execution, and future actors or alternative defensive systems would still require similar casting checks or plumbing adjustments.

Inheritance Restriction (Potential Extensibility Trade-off)

Since armour capability is tied to ArmableActor, any actor must extend it to gain armour support. If a future actor already extends another base class, composition or an armour component system may be needed. This can be mitigated later by extracting a composition-based equipment component if complexity grows.

Connascence Analysis:

Connascence of Name

Shared method names such as `equip()`, `unequip()`, `calculateDamage()`, and `isBroken()` establish a uniform contract between `Armour` and `ArmableActor`.

This ensures all armour integrates cleanly with the equipment system and damage-handling flow.

Connascence of Type

Shared abstract types such as `Armour`, `ArmableActor`, and `Equipable` form the core of the system.

Actions and combat logic depend on these abstractions instead of concrete armour types, aligning with DIP and supporting polymorphic behaviour.

Connascence of Meaning

The return value of `calculateDamage()` represents the *final damage applied to HP after armour mitigation*.

This value must always be interpreted consistently across armour types; misinterpreting it could result in incorrect damage or durability behaviour.

Connascence of Execution Order

Within the damage processing sequence:

1. `calculateDamage()` consumes durability and computes residual damage
2. `applyEffect()` triggers special effects (e.g., thorns) if damage penetrates
3. `isBroken()` determines armour status
4. If broken, armour is unequipped and removed
5. Remaining damage is applied to HP

Changing this order risks incorrect durability handling, effects triggering at the wrong time, or armour never breaking.

Conclusion

Finally, design is selected to implement `Armour` system, which uses a clean abstraction-based structure.

`ArmableActor` centralises armour-wearing ability, avoiding duplicated logic across actors.

The `Armour` hierarchy encapsulates damage reduction and special effects, while equip actions remain decoupled by operating only on abstractions.

Overall, this design supports future expansion of armour behaviours and wearable characters, improves testability through polymorphic interfaces, and integrates seamlessly into the engine while maintaining clarity and modularity.